Estudo de Caso A1 — O Problema da Entrega Inteligente

Otimização Logística em Plataformas Digitais de Delivery

Unidade Curricular: Estruturas de Dados e Análise de Algoritmos (0006963)
Professor: Alexandre "Montanha" de Oliveira
Nível: Graduação – Preparatório para a Avaliação A1
Modalidade: Análise e Raciocínio (sem implementação de código)
Formato de entrega: Documento Markdown ou PDF, estruturado e justificado

---

📚 Contextualização

Você é consultor técnico de uma startup de tecnologia contratado pela **FastBite**, uma plataforma de delivery que opera em três capitais brasileiras com mais de **80.000 pedidos por dia**. A empresa enfrenta um desafio crítico: o sistema atual de atribuição de pedidos e definição de rotas é lento, impreciso e gera reclamações frequentes por parte de clientes e entregadores.

A diretora de engenharia convocou seu time para propor uma solução computacionalmente sólida. Antes de qualquer implementação, ela quer **entender o problema com profundidade técnica**: que tipo de problema é esse? Qual é o custo de resolvê-lo de forma ótima? Quais estratégias são viáveis no mundo real?

O ambiente operacional da FastBite

A cada ciclo de **30 segundos**, o sistema recebe um lote de pedidos. Para cada pedido, são conhecidos:

- **localização do restaurante** (ponto de coleta)
- **localização do cliente** (ponto de entrega)
- **tempo estimado de preparo** do restaurante
- **prioridade do pedido** (padrão, premium ou urgente)
- **peso e volume** da entrega (relevante para capacidade da mochila do entregador)

Para cada entregador disponível, são conhecidos:

- **localização atual** (coordenadas GPS em tempo real)
- **capacidade máxima de carga** (quantidade de pedidos simultâneos)
- **tipo de veículo** (bicicleta, moto, carro — cada um com velocidade média e custo diferente)
- **histórico de avaliações** (entregadores bem avaliados têm preferência em clientes premium)

As restrições operacionais incluem:

- Um entregador pode transportar **até 3 pedidos simultâneos**
- Pedidos urgentes devem ser entregues em **menos de 20 minutos**
- O sistema deve tomar decisões em **tempo real** — nenhuma decisão pode levar mais de **2 segundos** de processamento

- Condições de trânsito variam ao longo do dia e afetam os tempos estimados

---

💧 Problema Central

A FastBite precisa responder, a cada ciclo de 30 segundos, à seguinte pergunta:

> **Como atribuir pedidos aos entregadores disponíveis e definir a ordem de coleta/entrega de forma a minimizar o tempo total de entrega, respeitando todas as restrições operacionais?**

Este desafio envolve **duas decisões combinadas e interdependentes**:

1. **Atribuição:** qual entregador recebe qual conjunto de pedidos?
2. **Roteamento:** em qual ordem o entregador deve coletar e entregar esses pedidos?

A combinação dessas decisões resulta em um espaço de soluções que cresce de forma **explosiva** conforme o número de pedidos e entregadores aumenta. Com apenas 10 pedidos e 5 entregadores, o número de combinações possíveis já supera dezenas de milhares. Com 50 pedidos e 20 entregadores — cenário típico em horário de pico —, a avaliação exaustiva de todas as soluções possíveis se torna computacionalmente inviável.

Esse problema é diretamente relacionado a dois problemas clássicos da teoria da computação:

- **Problema do Caixeiro Viajante (TSP — Travelling Salesman Problem):** dado um conjunto de cidades e distâncias entre elas, encontrar o menor circuito que visita todas as cidades exatamente uma vez.
- **Problema de Roteamento de Veículos (VRP — Vehicle Routing Problem):** generalização do TSP com múltiplos veículos, restrições de capacidade e janelas de tempo.

Ambos são amplamente estudados e pertencem a uma categoria especial de problemas na teoria da complexidade computacional.

⚠ Complexidade Computacional

Questão 1 — Classificação do Problema (20 pontos)

Com base no contexto apresentado, responda:

a) O problema de roteamento da FastBite pertence à classe **P**, **NP** ou **NP-Completo**? Justifique sua resposta com base na definição formal de cada classe.

> Considere as definições:
>
> - **P:** problemas resolvidos por algoritmo determinístico em tempo polinomial.
> - **NP:** problemas cuja solução pode ser **verificada** em tempo polinomial.
> - **NP-Completo:** problemas em NP que são tão difíceis quanto qualquer outro problema em NP (todo problema NP se reduz a eles em tempo polinomial).

b) Demonstre, de forma intuitiva, que o problema da FastBite pode ser **reduzido** ao TSP. Para isso, descreva como transformar uma instância do problema de roteamento da FastBite em uma instância do

TSP. (Não é necessária prova formal — uma descrição clara e fundamentada é suficiente.)

c) Por que a solução por **força bruta** é inviável para o problema da FastBite em produção? Calcule ou estime a quantidade de permutações possíveis de entregas para um cenário com:

- 8 pedidos
- 3 entregadores (cada um com até 3 pedidos)

Mostre o raciocínio combinatório e indique a complexidade assintótica resultante.

🧠 Tarefas de Análise e Proposta de Solução

Questão 2 — Abordagem Gulosa (Greedy) (25 pontos)

A equipe júnior da FastBite propôs o seguinte algoritmo para atribuição de pedidos:

> "Para cada pedido não atribuído, escolha o entregador disponível mais próximo do restaurante e atribua o pedido a ele. Repita até que todos os pedidos estejam atribuídos. Para a rota de cada entregador, ordene as entregas sempre indo ao ponto mais próximo do atual."

a) Descreva, em linguagem natural (sem código), o funcionamento desse algoritmo passo a passo para o cenário de exemplo da seção **Dados do Cenário**.

b) Explique por que esse algoritmo é classificado como **guloso (greedy)**. Qual é a propriedade de escolha local que ele aplica?

c) Apresente um **contraexemplo concreto** (pode ser hipotético ou adaptado do cenário) em que a solução gulosa leva a um resultado **subótimo** — ou seja, em que existiria uma atribuição melhor que o algoritmo guloso não encontraria.

d) Qual é a complexidade de tempo desse algoritmo guloso em função de `n` pedidos e `m` entregadores? Justifique.

Questão 3 — Programação Dinâmica e Divisão e Conquista (25 pontos)

a) A equipe sênior sugeriu aplicar **Programação Dinâmica (PD)** para resolver o problema de roteamento de cada entregador isoladamente (assumindo que a atribuição já está feita).

- Explique se PD é **aplicável** ao problema de roteamento de um único entregador com `k` pedidos.
- Defina informalmente o **subproblema** que a PD deveria resolver.
- Explique qual é o custo de memória e tempo dessa abordagem.
- Indique a partir de qual valor de `k` esta solução se torna impraticável em tempo real.

b) Avalie a aplicabilidade de **Divisão e Conquista** ao problema de roteamento da FastBite:

- É possível dividir o problema em subproblemas independentes? Em que condições?
- Como a **divisão geográfica da cidade** (por zonas ou quadrantes) poderia ser explorada como estratégia de Divisão e Conquista?

- Quais são as **limitações** dessa abordagem neste contexto? O que acontece nas fronteiras entre zonas?

Questão 4 — Comparação das Abordagens (15 pontos)

Preencha e desenvolva a tabela comparativa abaixo, considerando o contexto operacional da FastBite (decisão em até 2 segundos, milhares de pedidos por dia):

Critério	Greedy	Programação Dinâmica	Divisão e Conquista
Qualidade da solução			
Complexidade de tempo			
Complexidade de espaço			
Viabilidade em tempo real (≤ 2s)			
Escalabilidade com aumento de n			
Facilidade de adaptação a mudanças			

Após a tabela, escreva um parágrafo de **análise crítica** comparando as abordagens e justificando qual seria a mais adequada para uso como solução principal da FastBite, e por quê.

Questão 5 — Solução de Engenharia Real (10 pontos)

> "Na prática, empresas como iFood, Uber Eats e Rappi não resolvem o problema de forma ótima. Elas o resolvem de forma suficientemente boa, suficientemente rápida."

a) Explique, com suas palavras, o que é uma **heurística** no contexto de algoritmos. Por que heurísticas são preferíveis a soluções ótimas em sistemas como a FastBite?

b) Descreva, em linhas gerais, como uma **solução de engenharia real** para a FastBite poderia ser estruturada. Considere a combinação de:

- uma fase de **particionamento** dos pedidos por região geográfica
- uma heurística **gulosa** como solução inicial
- uma etapa de **refinamento local** (por exemplo, trocar dois pedidos de dois entregadores e verificar se melhora)
- um limite de tempo estrito para interrupção do processamento

c) Quando vale a pena **buscar a solução ótima**? Dê um exemplo de situação — mesmo dentro do contexto de delivery — em que calcular a solução exata seria razoável.

Questão 6 — Reflexão Crítica (5 pontos)

Responda em até 15 linhas:

> "Todo engenheiro de software enfrenta o dilema entre a solução ideal e a solução possível. No contexto de sistemas de larga escala, quando 'bom o suficiente' é, na verdade, a melhor decisão técnica?"

Fundamente sua resposta nos conceitos de complexidade computacional e nos trade-offs discutidos ao longo desta atividade.

📦 Dados do Cenário

Use os dados abaixo como apoio às suas respostas. Eles representam um **mini-ciclo** de atribuição da FastBite.

Pedidos disponíveis

Pedido	Restaurante	Cliente	Prioridade	Tempo de preparo
P1	Setor A (0, 2)	Setor B (4, 5)	Urgente	5 min
P2	Setor A (1, 1)	Setor C (7, 2)	Padrão	10 min
P3	Setor B (4, 4)	Setor A (0, 0)	Premium	8 min
P4	Setor C (6, 1)	Setor B (3, 6)	Padrão	12 min
P5	Setor B (5, 5)	Setor C (8, 1)	Urgente	3 min

> **Distância:** use distância Manhattan — `|x1 - x2| + |y1 - y2|`

Entregadores disponíveis

Entregador	Posição atual	Capacidade	Veículo
E1	(1, 1)	2 pedidos	Bicicleta
E2	(5, 3)	2 pedidos	Moto
E3	(7, 6)	3 pedidos	Moto

> Use esses dados para ilustrar suas respostas quando solicitado. Não é necessário resolver o problema de forma exaustiva.

📊 Critérios de Avaliação

Critério	Peso
Classificação correta do problema (P/NP/NP-C) com justificativa	20%
Qualidade da análise da abordagem gulosa e contraexemplo	25%
Aplicabilidade de PD e Divisão e Conquista	25%

Critério	Peso
Análise comparativa e tabela	15%
Solução de engenharia real e reflexão crítica	15%

A avaliação considera, transversalmente, em todas as questões:

- clareza e precisão da linguagem técnica
- coerência entre os conceitos e os exemplos utilizados
- profundidade do raciocínio e consistência das justificativas
- organização e estrutura do documento entregue

📤 Entregável

- **Formato:** Markdown (`.md`) ou PDF
- **Estrutura obrigatória:** título, identificação, respostas numeradas por questão
- **Extensão recomendada:** entre 4 e 8 páginas (ou equivalente em Markdown)
- **Código:** não é necessário nem esperado
- **Entrega via:** repositório pessoal no GitHub (link enviado pelo Ulife) ou upload direto no Ulife

---

💡 Dicas do Professor

> Estas orientações não revelam as respostas — elas apontam o caminho do raciocínio correto.

- **Sobre complexidade:** pense em como o número de combinações cresce quando você adiciona mais um pedido ou mais um entregador. Dobre o número de pedidos — o que acontece com o número de rotas possíveis?

- **Sobre força bruta:** antes de dizer que é inviável, calcule. Mostre os números. Um argumento quantitativo vale muito mais do que uma afirmação genérica.

- **Sobre o greedy:** lembre-se de que um algoritmo guloso toma a melhor decisão *local* a cada passo. Mas o que parece ótimo agora pode fechar portas para uma solução global melhor. Consegue pensar em um caso assim?

- **Sobre Programação Dinâmica:** PD é poderosa quando um problema tem *subestrutura ótima* e *sobreposição de subproblemas*. O roteamento de um entregador com `k` paradas tem essa propriedade? Pense no TSP com memoização — e em qual é o custo de memória.

- **Sobre a solução real:** pense como engenheiro, não como matemático. Uma empresa com 80.000 pedidos por dia não quer a rota perfeita — quer uma rota *boa o suficiente entregue em 2 segundos*. O que você faria?

- **Sobre "bom o suficiente":** há uma diferença entre *não conseguir* a solução ótima e *não precisar* dela. Saber distinguir esses casos é uma habilidade essencial de engenharia.

- **Sobre o cenário:** os dados fornecidos são simples de propósito. Use-os para ilustrar seu raciocínio, não para fazer cálculos exaustivos. O objetivo é mostrar que você entende os conceitos, não que você consegue somar distâncias.

🎯 Objetivo de Aprendizagem

Ao concluir esta atividade, espera-se que você seja capaz de:

1. **Identificar** classes de complexidade e classificar problemas reais com justificativa técnica
2. **Relacionar** problemas práticos a modelos teóricos consagrados (TSP, VRP)
3. **Avaliar** trade-offs entre qualidade de solução e custo computacional
4. **Propor** estratégias algorítmicas adequadas ao contexto e às restrições do problema
5. **Tomar decisões** técnicas fundamentadas, como faz um engenheiro de software em ambiente produtivo

> *"A ciência da computação não é sobre computadores tanto quanto a astronomia não é sobre telescópios."*
> — Edsger W. Dijkstra